Kigali Clinic: Quiz 2, Completing the Backend

Web Technology and Internet · AUCA · Functional requirements · Duration: 4 hours · Individual work · 100 marks + 10 bonus

1. Context

In Quiz 1 you practised reading data with derived queries and JPQL on the clinic API (Patient, Doctor, Office, Specialization and Appointment with the AppointmentStatus enum). A real clinic needs more than CRUD and search: the receptionist must manage one specialization at a time and move doctors between offices, appointments must follow business rules, the doctor must record prescriptions after a consultation, and the staff need an agenda and a dashboard. In this quiz you add 15 of these features, including one new entity (Prescription).

2. Rules

- Continue in your own clinic project from Quiz 1, with the same database.
- Every requirement is a full feature: repository method (when a query is needed), service method and a controller endpoint with the exact path, exact message and HTTP status given below, tested in Postman.
- Keep the class style: entities are used directly as request and response bodies, services return a List, an Optional or a String message, and the controller maps the message to an HttpStatus with if/else. No DTOs, no try/catch. (Only D4 returns a Map.)
- A requirement marked DERIVED must be solved by method name only. A requirement marked JPQL must use `@Query`. Using the other type scores zero for that requirement.
- Spring Data documentation is allowed. Classmates, chat tools and copied repositories are not.
- Commit after each part. The commit history is part of your proof of work.

3. Step 0: prepare your data (first 15 minutes)

Start from your Quiz 1 data, then make sure you have at least:

- Office: 4 offices (101, 102, 103, 104). Office 104 has no doctor.
- Doctor: 3 doctors, each with an office. One of them has only one specialization.
- Patient: 5 patients. Note the ssNumber of one patient; you will use it a lot today.
- Appointment: 3 appointments dated from today onward (e.g. 2026-10-20, 2026-10-22, 2026-10-30) for that patient, with status SCHEDULED or CONFIRMED. On 2026-10-20 the same doctor also has one CANCELLED appointment.
- At least 2 appointments with status COMPLETED. Prescriptions in Part C can only be written for them.

4. Part A: doctors and offices (20 marks, 5 each, about 40 minutes)

ID	Requirement	Endpoint	Expected behaviour	Query type
A1	Add one specialization to a doctor	`PUT /api/doctors/{doctorId}/specializations/{specializationId}`	Adds the specialization to the doctor's list and keeps the ones he already has. 404 "The doctor with that id does not exist" or "That specialization does not exist". 409 "The doctor already has that specialization". 200 "Specialization added to the doctor".	Many-to-many: list add, contains, save the owner (Doctor)
A2	Remove one specialization from a doctor	`DELETE /api/doctors/{doctorId}/specializations/{specializationId}`	Removes only the link in *doctor_specialization*; the specialization itself stays. Same two 404 messages as A1. 409 "The doctor does not have that specialization". 200 "Specialization removed from the doctor".	Many-to-many: list remove, save

ID	Requirement	Endpoint	Expected behaviour	Query type
A3	Free offices	`GET /api/offices/free`	Returns every office that no doctor occupies, sorted by office number. With the seed data this returns office 104.	JPQL (NOT EXISTS subquery or LEFT JOIN)
A4	Move a doctor to another office	`PATCH /api/doctors/{doctorId}/office?officeNumber=104`	Gives the doctor the new office; the old office becomes free (check with A3). 404 "The doctor with that id does not exist" or "The office with that office number does not exist". 409 "That office is taken by other doctor". 200 "Office changed successfully".	Business rule + DERIVED (existsBy)

5. Part B: appointment rules (25 marks, about 55 minutes)

The status of an appointment may only move along these arrows. COMPLETED and CANCELLED are final.

Current status	Allowed new status	Anything else
SCHEDULED	CONFIRMED or CANCELLED	400 "Cannot change status from X to Y"
CONFIRMED	COMPLETED or CANCELLED	400 "Cannot change status from X to Y"
COMPLETED, CANCELLED	none	400 "Cannot change status from X to Y"

ID	Requirement	Endpoint	Expected behaviour	Query type
B1 (5)	Safe booking	`POST /api/appointments/save (modify the existing endpoint)`	If the appointment date is before today, do not save; return 400 "Appointment date cannot be in the past". A new appointment always starts as SCHEDULED, whatever status the JSON body contains.	Business rule (LocalDate.now(), isBefore)
B2 (10)	Status flow	`PATCH /api/appointments/{id}/status?status=COMPLETED (or modify your existing status endpoint)`	Accept only the moves in the table above. Otherwise return 400 with the real names, e.g. "Cannot change status from COMPLETED to SCHEDULED". 404 "The appointment with that id does not exist". 200 "Appointment status updated successfully". If your general PUT update can also change the status, stop it, so the status only changes here.	Business rule (enum comparison with ==)
B3 (10)	Reschedule an appointment	`PATCH /api/appointments/{id}/reschedule?newDate=2026-10-30`	404 if the id does not exist. 400 "A COMPLETED appointment cannot be rescheduled" (or CANCELLED). 400 "Appointment date cannot be in the past". 409 "Doctor is already booked on that date" when the doctor has a non-CANCELLED appointment on newDate. On success set the new date, set the status back to SCHEDULED and return 200 "Appointment rescheduled successfully".	DERIVED (existsBy, And, StatusNot) + rules

6. Part C: prescriptions, a new entity (30 marks, about 70 minutes)

After a consultation the doctor writes one or more prescriptions. Create the entity Prescription (table `prescription`) with: `UUID id`, `String medicineName`, `String dosage` (e.g. "1 tablet, 3 times a day"), `int durationDays`, `LocalDate issuedDate` and `@ManyToOne Appointment appointment` (column `appointment_id`). One appointment can have many prescriptions. Create PrescriptionRepository, PrescriptionService and PrescriptionController.

ID	Requirement	Endpoint	Expected behaviour	Query type
C1 (10)	Write a prescription	`POST /api/prescriptions/save?appointmentId=...`	Body: {"medicineName":"Amoxicillin", "dosage":"1 tablet, 3 times a day", "durationDays":7}. 404 "The appointment with that id does not exist". 400 "Prescriptions are only for COMPLETED appointments". The service sets issuedDate to today and links the appointment. 201 "Prescription saved successfully".	New entity, @ManyToOne, rule
C2 (5)	Prescriptions of an appointment	`GET /api/prescriptions/appointment/{appointmentId}`	Returns the prescriptions of that appointment. 404 "The appointment with that id does not exist". Empty list when it has none.	DERIVED (findBy entity)
C3 (5)	Prescription history of a patient	`GET /api/prescriptions/patient/{ssNumber}`	Returns every prescription of that patient across all appointments, newest issuedDate first. Empty list when there is none.	JPQL (path navigation p.appointment.patient)
C4 (10)	Update, delete, protect	`PUT /api/prescriptions/update/{id}` `DELETE /api/prescriptions/delete/{id}` `+ your appointment DELETE endpoint (modify)`	Update changes only medicineName, dosage and durationDays: 200 "Prescription updated successfully". Delete: 200 "Prescription deleted successfully". Both: 404 "The prescription with that id does not exist". An appointment that has prescriptions can no longer be deleted: 409 "That appointment has prescriptions, delete them first".	CRUD + DERIVED (existsBy)

7. Part D: agenda and reports (25 marks, about 45 minutes)

ID	Requirement	Endpoint	Expected behaviour	Query type
D1 (5)	Doctor's agenda for a day	`GET /api/doctors/{doctorId}/agenda?date=2026-10-20`	Returns that doctor's appointments on that date, without the CANCELLED ones. 404 "The doctor with that id does not exist".	DERIVED (And, StatusNot)
D2 (5)	Upcoming appointments of a patient	`GET /api/patients/{ssNumber}/upcoming`	Returns the patient's appointments dated today or later whose status is SCHEDULED or CONFIRMED, soonest first. 404 "That patient with that ssnumber does not exist".	DERIVED (GreaterThanEqual, In, OrderBy)
D3 (5)	Search doctors by name	`GET /api/doctors/search?name=gis`	Returns the doctors whose first name or last name contains the text, ignoring letter case, sorted by last name. e.g. "gis" finds a doctor named Mugisha.	JPQL (LIKE, LOWER, OR)
D4 (10)	Clinic dashboard	`GET /api/dashboard/summary`	Create DashboardService and DashboardController. Return one JSON object, e.g. {"patients":5, "doctors":3, "freeOffices":1, "appointmentsToday":0, "scheduled":4, "prescriptions":6}. The service builds a Map<String, Long> (use LinkedHashMap to keep the order).	count(), DERIVED countBy, reuse A3

8. Bonus (up to +10, only after all 15 requirements work)

ID	Requirement	Endpoint	Expected behaviour	Query type
+5	Most prescribed medicines	`GET /api/prescriptions/stats/top-medicines`	One row per medicine with how many times it was prescribed, most prescribed first, e.g. [["Amoxicillin",3],["Paracetamol",2]], as List<Object[]>.	JPQL (GROUP BY, COUNT, ORDER BY)
+5	Doctor workload	`GET /api/doctors/workload?start=2026-10-01&end=2026-10-31`	For each doctor: first name, last name and number of COMPLETED appointments between the two dates (inclusive), busiest doctor first. Doctors with zero do not appear.	JPQL (JOIN, WHERE, BETWEEN, GROUP BY)

9. Hints

- A many-to-many link is stored by the owning side, the entity with @JoinTable (Doctor). Change its list and save it: `doctor.getSpecializations().add(spec); doctorRepository.save(doctor);` Use `contains` and `remove` on the same list.
- To change an office, set it on the entity that has the @JoinColumn (Doctor) and save that entity.
- Offices without a doctor, with a subquery: `SELECT o FROM Office o WHERE NOT EXISTS (SELECT d FROM Doctor d WHERE d.office = o)`
- Dates: `LocalDate.now()`, `date.isBefore(LocalDate.now())`. Enums are compared with `==`, e.g. `status == AppointmentStatus.COMPLETED`.
- New derived keywords today: `StatusNot`, `GreaterThanEqual`, `StatusIn` (takes a List), `countBy...` (returns long). JpaRepository already gives you `count()`.
- Single-valued relations are navigated with a dot, no JOIN needed: `WHERE p.appointment.patient.ssNumber = :ss`
- The Prescription JSON will show its appointment, patient and doctor. Do not add a List<Prescription> to Appointment; if you do, mark it @JsonIgnore to avoid an endless JSON loop.
- Spring matches fixed paths like `/search` or `/free` before `/{id}`, so their order in the controller does not matter.
- Keep spring.jpa.show-sql=true. When a query returns nothing, read the generated SQL in the console.

10. Marking and timing

Part	Requirements	Marks	Suggested time
Step 0: data	-	-	15 min
A: doctors and offices	A1 to A4 (5 each)	20	40 min
B: appointment rules	B1 (5), B2 (10), B3 (10)	25	55 min
C: prescriptions	C1 (10), C2 (5), C3 (5), C4 (10)	30	70 min
D: agenda and reports	D1 (5), D2 (5), D3 (5), D4 (10)	25	45 min
Submission and demo	-	-	15 min
Bonus	2 requirements (5 each)	+10	if time remains

Inside each requirement: 60% for correct logic (the query of the required type returns the right rows, or the business rule blocks exactly the right cases), 25% for an endpoint that follows the specification (path, parameters, messages, HTTP status), 15% for showing it working in Postman during the live demo, including at least one failing case (404, 400 or 409). A requirement using the wrong query type scores 0.

11. Submission

- Push your final commit to your GitHub repository before the last 15 minutes of the session.
- Export your Postman collection as JSON and add it to the root of the repository as `clinic-quiz2.postman_collection.json`. Put one request per requirement, named by its ID (A1, B2, ...).
- Live demo: the lecturer picks two requirements at random. You run them in Postman, show one success and one error response, and explain your code in one or two sentences.